

Roadmap técnico

Da consolidação arquitetural à inteligência de código local

TESE

O próximo salto do Nocturne não depende de adicionar mais superfície de IA. Depende de transformar a base atual em um sistema mais previsível, observável e capaz de entender mudanças de código com segurança.

Documento baseado na análise estática descrita na conversa de referência. As janelas são relativas e servem para ordenar dependências, não para fixar compromissos de calendário.

29 de agosto de 2026

O roadmap em uma página

O projeto já apresenta fundamentos fortes: separação entre renderer e processo principal, APIs IPC tipadas, isolamento do Electron, persistência local e uma organização por domínios que favorece evolução. O principal risco é o crescimento de pontos de coordenação centrais, especialmente no renderer e no registro de IPC.

PRINCÍPIO

Consolidar primeiro, tornar o código observável depois, e só então ampliar a inteligência. Cada fase deve deixar um contrato reutilizável para a seguinte.

MAPA DE FASES

FASE	PRIORIDADE	FOCO	SAÍDA PRINCIPAL
1 Consolidação arquitetural	P0	Reduzir acoplamento e custo de renderização	Contratos claros + renderer mais previsível
2 Code Intelligence	P1	Construir uma visão estruturada do projeto	Índice incremental + validação confiável
3 Controle de mudanças	P0/P1	Tornar alterações auditáveis e reversíveis	Diff, aprovação, checkpoints e histórico
4 Inteligência avançada	P2/P3	Aumentar contexto e qualidade das sugestões	Busca semântica, grafo e automações maduras

ORDEM RECOMENDADA

1. Estabilizar contratos

Começar por App.tsx, Database.ts e registerIpc.ts. Separar orquestração, tipar o lifecycle do agente e reduzir rerenders antes de acrescentar novas capacidades.

2. Indexar o projeto

Criar a base de Code Intelligence: arquivos, stack, símbolos e imports. O índice deve ser incremental, cancelável e observável.

3. Controlar mudanças

Colocar diff, aprovação por hunk, checkpoints e histórico de execução no centro da experiência de agente.

4. Expandir inteligência

Só depois ativar busca semântica local, grafo de dependências, sugestões arquiteturais e, por último, cenários multi-agent.

Menos acoplamento, contratos explícitos e um renderer mais previsível.

OBJETIVO

Reduzir o risco de evolução concentrado nos pontos que hoje coordenam muitos fluxos: App.tsx, persistência, registro de IPC e lifecycle do agente.

POR QUE AGORA

- App.tsx acumulou responsabilidades de workspace, conversas, mensagens, attachments, settings, Git, previews, memory, Agent Mode, retry, scroll, atalhos e eventos do agente.
- Database.ts e registerIpc.ts são pontos naturais de crescimento: sem fronteiras explícitas, cada novo domínio aumenta o custo de mudança e de teste.
- O renderer já possui domínios e hooks suficientes para permitir extração gradual, sem uma reescrita total.

ENTREGÁVEIS

ENTREGÁVEL	CONTEÚDO PROPOSTO	RESULTADO ESPERADO
App.tsx por fluxo	Extrair providers, hooks e componentes de workspace, conversa, agente, preview, memory e settings. Manter App.tsx como composição de alto nível.	Menos estado compartilhado e menor custo cognitivo por mudança.
Database.ts modular	Separar conexão, migrações, repositórios por domínio e transações. Adicionar contratos de erro e testes de recuperação.	Persistência mais testável e migrações com menor risco.
IPC por domínio	Organizar registerIpc.ts por módulos e registrar handlers com schemas de entrada e saída. Eliminar handlers genéricos sem contrato.	Superfície IPC legível, auditável e mais fácil de evoluir.
Lifecycle do agente tipado	Modelar estados, eventos, cancelamento, retry, autorização e término em uma máquina de estados ou união discriminada.	Fluxos de execução previsíveis e telemetria coerente.
Rerenders e observabilidade	Revisar seletores Zustand, subscriptions, memoização, listas de mensagens e eventos de filesystem. Instrumentar render, IPC e consultas lentas.	Interações mais estáveis com regressões mensuráveis.

CRITÉRIO DE SAÍDA

- App.tsx não concentra regras de negócio; coordena composição e navegação.
- Cada novo domínio possui contrato, testes e ponto de entrada identificado.
- Cancelamento, retry e erro do agente podem ser reproduzidos em testes.

PRIORIDADE DE IMPLEMENTAÇÃO

1. Tipos e contratos compartilhados.
2. Extração de App.tsx por fatias verticais.
3. Modularização de Database.ts e IPC.
4. Métricas de rerender, IPC e persistência.

OBJETIVO

Fazer o Nocturne entender o projeto como um conjunto de arquivos, símbolos, imports, tecnologias e resultados de validação, com atualização incremental e rastreável.

ESCOPO TÉCNICO

ENTREGÁVEL	CONTEÚDO PROPOSTO	RESULTADO ESPERADO
Project Index	Criar um índice por workspace com arquivos relevantes, hash, tamanho, linguagem, timestamps, exclusões e estado de processamento. Atualizar por eventos de filesystem e permitir reindexação segura.	Visão consistente do workspace, sem depender de varreduras completas a cada pergunta.
Símbolos e imports	Extrair símbolos por linguagem suportada e relações de import/export. Persistir localização, assinatura, escopo e referências suficientes para navegação e contexto.	Contexto de código mais preciso e explicável.
Deteccção de stack	Inferir linguagens, frameworks, gerenciadores, scripts, testes, build e convenções a partir de arquivos de configuração e estrutura do projeto.	Prompts e validações adaptados ao projeto real.
Validation Pipeline	Orquestrar typecheck, lint, testes, build e smoke checks por perfil. Capturar comando, duração, exit code, saída resumida e artefatos.	Feedback pós-mudança uniforme, com diagnóstico acionável.
Status e recuperação	Expor progresso, cancelamento, fila, falha parcial e reprocessamento. Evitar que um arquivo problemático invalide todo o índice.	Operação local robusta e observável.

DECISÕES QUE EVITAM RETRABALHO

- Começar com um modelo agnóstico de linguagem e adapters por parser; não acoplar o índice ao provedor de IA.
- Persistir referências ao arquivo e ao hash analisado para que qualquer resposta possa explicar de qual estado veio.
- Separar descoberta, indexação e validação: são pipelines diferentes, com custos e falhas diferentes.
- Definir limites de escopo, exclusões e privacidade antes de introduzir embeddings ou sincronização externa.

CRITÉRIO DE SAÍDA

- Abrir um workspace produz um status de indexação visível e cancelável.
- Uma mudança de arquivo atualiza apenas o necessário.
- O Validation Pipeline retorna resultado estruturado e pode ser associado a uma execução.

PRIORIDADE

P1, com uma fatia mínima antecipada em P0: descoberta de stack e execução estruturada de validações já aumentam a segurança das fases seguintes.

Transformar a execução do agente em um fluxo revisável, aprovável e reversível.

OBJETIVO

Dar ao usuário controle granular sobre o que será alterado, por que será alterado, qual validação foi executada e como voltar a um estado anterior.

ENTREGÁVEIS

ENTREGÁVEL	CONTEÚDO PROPOSTO	RESULTADO ESPERADO
Diff por arquivo	Representar alterações por arquivo com estado novo, estado anterior, status de validação e origem da mudança. Permitir agrupar por execução.	O usuário entende o impacto antes de aprovar.
Aprovação por hunk	Permitir aceitar, rejeitar ou editar blocos de mudança. Tratar conflitos e dependências entre hunks com avisos claros.	Aprovação granular sem perder rastreabilidade.
Checkpoints	Criar snapshots antes de executar alterações e após validações bem-sucedidas. Associar checkpoint ao workspace e à execução.	Rollback simples e seguro.
Histórico de execuções	Persistir prompt, intenção, arquivos tocados, diffs, comandos de validação, resultados, duração, erros e decisão do usuário.	Auditoria local e base para aprendizado do produto.
Políticas de segurança	Definir áreas protegidas, arquivos sensíveis, comandos que exigem confirmação e comportamento em workspace não autorizado.	Menos surpresas e maior confiança no Agent Mode.

FLUXO RECOMENDADO

1. Intenção	2. Planejamento	3. Mudança	4. Validação	5. Decisão
Prompt e contexto	Arquivos e ações previstas	Patch por arquivo	Pipeline estruturado	Aprovar, rejeitar ou reverter

CRITÉRIO DE SAÍDA

- Nenhuma alteração automática precisa ser aceita como um pacote indivisível.
- Toda execução pode ser localizada, comparada e revertida.
- O resultado da validação aparece junto da mudança, não em uma tela separada sem contexto.

ORDEM INTERNA

- 1. Checkpoint e histórico mínimo.
- 2. Diff por arquivo.
- 3. Aprovação por hunk.
- 4. Políticas e conflitos.

Adicionar contexto sem perder privacidade, explicabilidade e controle operacional.

OBJETIVO

Usar o índice, o histórico e as validações acumuladas para aumentar a qualidade das respostas e sugestões, mantendo as capacidades avançadas subordinadas a sinais concretos do projeto.

ENTREGÁVEIS

ENTREGÁVEL	CONTEÚDO PROPOSTO	RESULTADO ESPERADO
Busca semântica local	Gerar embeddings apenas dentro do workspace, com escopo configurável, exclusões, reindexação e explicação dos trechos recuperados.	Perguntas encontram contexto relevante sem enviar todo o projeto ao provedor.
Grafo de dependências	Modelar relações entre arquivos, símbolos, pacotes, scripts e testes. Exibir impacto provável de uma mudança e ciclos relevantes.	Planejamento e revisão com visão de impacto.
Sugestões arquiteturais	Detectar sinais como duplicação, acoplamento, handlers sem contrato, módulos centrais superdimensionados e validações ausentes. Sempre apresentar evidência.	Recomendações acionáveis, priorizadas e auditáveis.
Multi-agent somente depois	Adicionar agentes especializados apenas quando contratos, checkpoints, histórico, indexação e validação forem estáveis. Começar por papéis sequenciais e limites claros.	Paralelismo com governança, sem multiplicar a imprevisibilidade.

GUARDRAILS

- Privacidade local-first: o usuário decide o que pode ser indexado, vetorizado ou enviado.
- Explicabilidade: cada sugestão aponta para arquivos, símbolos, imports, histórico ou validações que a sustentam.
- Custo e latência: medir tamanho do índice, tempo de recuperação, tokens economizados e taxa de aceitação.
- Falha segura: se o índice estiver desatualizado ou a validação falhar, reduzir a confiança da sugestão e pedir revisão.

QUANDO LIBERAR MULTI-AGENT

GATE

Somente após existir um protocolo de tarefas, estado compartilhado limitado, checkpoint por agente, orçamento de execução, cancelamento e uma forma simples de explicar quem fez o quê.

Como saber se o roadmap está funcionando

O objetivo não é medir quantidade de features. É medir se o Nocturne se torna mais previsível para quem usa, mantém e amplia o produto.

INDICADORES SUGERIDOS

DIMENSÃO	SINAL ACOMPANHADO	DIREÇÃO DESEJADA
Renderer	Tempo de interação, renders por ação e regressões de estado	Menor variabilidade; sem regressões em fluxos centrais
IPC e persistência	Erros por domínio, latência de consultas e falhas de migração	Contratos explícitos e recuperação previsível
Code Intelligence	Cobertura do índice, tempo de atualização e taxa de falha parcial	Atualização incremental e estado visível
Mudanças	Execuções com checkpoint, rollback e validação associada	Toda mudança importante é auditável e reversível
IA	Aceitação de sugestões, contexto recuperado e custo por tarefa	Mais precisão com menos contexto desnecessário

SUGESTÃO DE ORDEM DE IMPLEMENTAÇÃO

SPRINTS INICIAIS

- Mapear responsabilidades e contratos de App.tsx.
- Extrair uma fatia vertical completa: estado, hook, componente e teste.
- Criar types compartilhados para lifecycle, IPC e resultados de validação.
- Adicionar métricas básicas de rerender, IPC e consultas.

APÓS A BASE

- Entregar Project Index mínimo e stack detection.
- Conectar Validation Pipeline ao histórico de execução.
- Construir checkpoints e diff antes de automatizar novas ações.
- Liberar busca semântica e grafo por incrementos mensuráveis.

RECOMENDAÇÃO O FINAL

Trate a Fase 1 como uma iniciativa de plataforma, não como limpeza de código. Ela reduz o custo de tudo que vem depois. O diferencial do Nocturne pode ser uma experiência de IA que conhece o projeto, mostra suas evidências e respeita a decisão do desenvolvedor.

Base: análise estática do repositório e observações registradas na conversa de referência. Este documento não substitui uma rodada de execução da suíte de testes, profiling e validação com usuários.